

```
-----
--
--                               JWT (Spring Security)
--
-----
```

```
>> What is JWT ?
```

```
-- JWT (JSON Web Token) is a token-based authentication mechanism used in REST APIs.
```

```
>> JWT is used when we want :
```

```
-- Stateless Authentication (no session stored on server)
-- Secure API access
-- Microservices friendly authentication
-- Frontend + Backend separated login system
```

```
=====
```

```
>> Why JWT is used ?
```

```
-- In session-based login, server stores session data.
-- In JWT, server does not store session, token itself contains user identity.
```

```
=====
```

```
>> Advantages
```

```
-- Stateless (No session stored)
-- Faster for APIs
-- Scalable (works well in microservices)
-- Works easily with React / Angular / Mobile apps
-- Token can be used across services
```

```
=====
```

```
>> JWT Structure (3 Parts) : JWT = Header , Payload , Signature
```

```
Example : eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyIiwiaWF0IjoxNjg4Nj...signature
```

```
=====
```

```
1. Header : Contains algorithm and token type
```

```
Example :
```

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

```
=====
```

```
2. Payload : Contains user details (claims)
```

```
Example :
```

```
{
  "sub": "user",
  "role": "ADMIN",
  "iat": 1710000000,
  "exp": 1710003600
}
```

```
=====
```

3. Signature : Ensures token is not tampered

Signature = HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)

--

>> Important JWT Terms

1. Claims

-- Information stored inside payload

Types of claims :

-- Registered claims (sub, exp, iat)

-- Public claims (custom like role)

-- Private claims (internal data)

2. Subject (sub)

-- Usually username

3. Issued At (iat)

-- Token created time

4. Expiration (exp)

-- Token expiry time

5. Secret Key

-- Used to sign and verify JWT

--

>> JWT Authentication Flow

=====

Step 1 : User sends login request : POST /auth/login

```
{
  "username": "user",
  "password": "1234"
}
```

=====

Step 2 : Spring Security authenticates username + password using
AuthenticationManager

=====

Step 3 : If login success → Server generates JWT token

=====

Step 4 : Server sends token to client : Response :

```
{
  "token": "Bearer eyJhbGciOiJIUzI1NiJ9..."
}
```

=====

Step 5 : Client stores token (frontend / mobile)

=====

Step 6 : For every next request, client sends token in header :

-- Authorization : Bearer <token>

=====

Step 7 : JwtFilter validates token and sets authentication in
SecurityContextHolder

=====

Step 8 : Spring Security checks authorization rules and allows/blocks API

--

>> JWT vs Session

=====

Feature	Session Based	JWT Based
-----	-----	-----
Stored on Server	Yes	No
Stateless	No	Yes
Scalable	Lo	High
Microservices	Not good	Best
Works with Mobile	Not good	Best
Logout	Easy	Needs token handling

--

>> Dependencies for JWT (Maven)

```
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-api</artifactId>
<version>0.11.5</version>
</dependency>
```

```
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-impl</artifactId>
<version>0.11.5</version>
</dependency>
```

```
<dependency>
<groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-jackson</artifactId>
<version>0.11.5</version>
</dependency>
```

--

>> JWT Utility Class (Generate + Validate Token)

=====

```
private final String SECRET = "mysecretkeymysecretkeymysecretkeymysecretkey";

public String generateToken(String username) {
    return Jwts.builder()
        .setSubject(username)
        .setIssuedAt(new Date())
        .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 *
60))
        .signWith(Keys.hmacShaKeyFor(SECRET.getBytes()),
SignatureAlgorithm.HS256)
        .compact();
}

public String extractUsername(String token) {
    return Jwts.parserBuilder()
        .setSigningKey(Keys.hmacShaKeyFor(SECRET.getBytes()))
        .build()
        .parseClaimsJws(token)
        .getBody()
        .getSubject();
}

public boolean validateToken(String token) {
    try {
        extractUsername(token);
        return true;
    } catch (Exception e) {
        return false;
    }
}
```

=====

1. SECRET Key
-- Used to sign JWT.
-- Must be strong and minimum 256-bit for HS256.
2. generateToken(username)
-- Creates JWT for user
-- Expires in 1 hour
3. extractUsername(token)
-- Reads token and returns subject (username)
4. validateToken(token)
-- Checks token validity (expiry + signature)

--

>> What does JwtAuthFilter do? (JWT Filter Work)

=====

```
@Component
public class JwtAuthFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtil jwtUtil;

    @Autowired
```

```

private UserDetailsService userDetailsService;

@Override
protected void doFilterInternal(HttpServletRequest request,
                                HttpServletResponse response,
                                FilterChain filterChain)
    throws ServletException, IOException {

    String authHeader = request.getHeader("Authorization");

    if (authHeader != null && authHeader.startsWith("Bearer ")) {

        String token = authHeader.substring(7);

        if (jwtUtil.validateToken(token)) {

            String username = jwtUtil.extractUsername(token);

            UserDetails userDetails =
userDetailsService.loadUserByUsername(username);

            UsernamePasswordAuthenticationToken authentication =
                new UsernamePasswordAuthenticationToken(
                    userDetails,
                    null,
                    userDetails.getAuthorities()
                );

SecurityContextHolder.getContext().setAuthentication(authentication);
        }

        filterChain.doFilter(request, response);
    }
}

```

=====

```

-- Extracts the token from the request header (Authorization : Bearer ...)
-- Validates the token (expiry + signature)
-- Extracts the username from the token
-- Loads the user from the database (UserDetailsService)
-- Creates an Authentication object
-- Sets it in SecurityContextHolder
-- Forwards the request to the next filter/controller

```

=====

>> Security Config for JWT

1. CSRF disabled (REST APIs)
2. SessionCreationPolicy.STATELESS
 - Server will not create session
 - Every request needs JWT
3. Permit login endpoint : /auth/**
4. JWT filter added before UsernamePasswordAuthenticationFilter

=====

>> SecurityContextHolder

-- Spring Security stores the currently logged-in (authenticated) user for the current request here.

-- When authentication is set in SecurityContextHolder then Spring considers the request as authenticated.

--

>> JWT Expiry and Refresh Token

1. Access Token

-- Short expiry (15 min - 1 hour)

2. Refresh Token

-- Long expiry (7 days - 30 days)

=====

>> Why refresh token ?

-- If access token expires quickly, user should not login again and again.

-- Refresh token is used to generate new access token.

Flow :

-- Login → accessToken + refreshToken

-- AccessToken expires → refreshToken generates new accessToken

--

>> JWT Logout (Important)

-- JWT is stateless, so logout is not automatic like sessions.

Options :

1. Client deletes token

-- Most common approach

2. Token blacklist in DB/Redis

-- Store invalid tokens

-- More secure

3. Short token expiry

-- Token auto expires quickly

--

>> JWT Exception Handling (401 / 403)

1. 401 Unauthorized

-- User not logged in / invalid token

2. 403 Forbidden

-- User logged in but not allowed for that endpoint

Best practice :

-- Use custom AuthenticationEntryPoint for 401

-- Use custom AccessDeniedHandler for 403

--

>> JWT Practices

-- Use strong secret key (minimum 256-bit)
-- Never store password inside token
-- Keep token expiry short
-- Use refresh token for long sessions
-- Use HTTPS in production
-- Validate token in every request
-- Store token safely in frontend
-- Do not expose secret key in GitHub

--

>> Final JWT Folder Structure

```
com.project
├── config
│   └── SecurityConfig.java
├── controller
│   └── AuthController.java
├── dto
│   ├── LoginRequest.java
│   └── LoginResponse.java
├── security
│   ├── JwtUtil.java
│   └── JwtAuthFilter.java
├── service
└── CustomUserDetailsService.java
```

=====

>> How to add JWT Roles / Authorities , Token Validation Expiry check

1. While generating JWT, add roles/authorities in claims

2. Send JWT in header: Authorization: Bearer <token>

3. In JwtAuthFilter, extract token from header

4. Validate token

-- signature verify

-- expiry check

5. Extract username + roles from token

6. Load user from DB using UserDetailsService

7. Create Authentication object with roles/authorities

8. Set Authentication in SecurityContextHolder

9. Continue request to controller (next filter)
